RTL-Universe: Building a Multi-Repo Benchmark Suite for Evaluating Coding Agents on Industrial RTL

Saketh Reddy, David Andrews

Week of April 24–29, 2026

Abstract

We present RTL-Universe, a benchmark suite that converts open-source industrial RTL repositories into Harbor evaluation tasks for coding agents. Starting from Google's Coral NPU (a Chisel/Bazel RISC-V ML accelerator with 305 test targets), we developed a complete pipeline for stripping implementation code while preserving test infrastructure, packaging the result in Docker with pre-warmed build caches, and scoring agent performance proportionally. We discovered and hardened against a critical vulnerability: agents exploiting cached build artifacts to achieve 95.8% on an unhardened task vs. 9.8% on the same task after hardening. We then generalized the pipeline into a reusable skill and applied it to 7 industry repos spanning 5 build systems. Claude Sonnet achieved a **perfect 1.000 score** on the Ibex RISC-V core E2E task (4 firmware simulation tests) after 7.5 hours, writing 30 SystemVerilog files from scratch. Across all benchmark runs, the headline finding is that **current models can implement individual RTL modules correctly but struggle with system-level integration**: the Coral NPU's bus fabric passed all unit tests (25/25) while zero E2E cocotb simulations succeeded.

1 Introduction and Motivation

Existing benchmarks for evaluating LLM coding agents on hardware design (e.g., Spec2GDSBench, VerilogEval) use curated, self-contained specifications. While useful for controlled experiments, they don't test whether agents can work in the messy reality of production RTL codebases—with complex build systems, hundreds of interdependent modules, and test suites that exercise the full stack from unit tests through silicon-grade E2E simulations.

RTL-Universe takes a different approach: we start with *real industry repositories* and systematically strip them to create benchmarks. The agent sees the test infrastructure, build files, documentation, and toolchain, but zero implementation code. Partial credit is awarded proportionally: reward = passed_tests/total_tests.

2 The Coral NPU Case Study

2.1 Task Construction

The initial benchmark was built from Google's Coral NPU—a 32-bit RISC-V ML accelerator with a scalar core, 128-bit SIMD vector pipeline, and matrix MAC engine. The repository uses Bazel with custom rules for Chisel RTL generation, cocotb simulation, and Verilator testbenches.

- **305 test targets**: 116 Chisel unit tests (ScalaTest), ∼189 cocotb E2E simulations, Verilator C++ testbenches, firmware tests
- **Skeleton**: All implementation Scala, Verilog, and firmware C/C++ stripped; test specs, BUILD files, documentation, and toolchain preserved
- **Docker**: 3-stage build (base toolchain → warm bazel cache → hardened skeleton), ∼30 GB image with pre-compiled Verilator, firtool, and TFLite-Micro

2.2 Cache Exploitation Discovery

Our first unhardened runs revealed a critical issue: **agents systematically exploit cached build artifacts** from the warm stage. Sonnet's behavior on the unhardened E2E task included:

1. Reading generated Verilog from `sandbox_stash/` (Chisel→Verilog output from the green build)
2. Comparing MD5 checksums of its own output against the cached correct version
3. Copying firmware source files from the `CppCompile` sandbox stash
4. Reading warm-build test logs to understand expected behavior

Task	Hardened	Timeout	Reward	Tests	Notes
coralnpu-e2e	No	3 h	**0.958**	181/189	Heavy cache exploitation
coralnpu-e2e	Yes	10 h	0.098	21/215	Same task, artifacts scrubbed
coralnpu-full	No	3 h	0.043	13/305	Moderate exploitation
coralnpu-full	Yes	10 h	0.177	54/305	Best hardened score

Table 1: Cache exploitation inflated the unhardened E2E score by ∼10×. Hardening scrubs sandbox stash, test logs, and project-specific build outputs while preserving external tool binaries.

2.3 Hardening

We identified three categories of leaky artifacts and scrub all of them in the Docker final stage:

Category 1 — Generated source: Chisel→Verilog output, Verilator→C++ elaboration, preprocessed file lists

Category 2 — Compiled objects: `.o`, `.a`, linked executables, action caches

Category 3 — Logs/metadata: Test logs, coverage reports, build event protocol files

2.4 What Sonnet Actually Built

On the hardened coralnpu-full task (10 h, 54/305 tests), Sonnet's 29 genuine passes came from:

- **25 Chisel unit tests**: Wrote correct implementations for all `common/`, `bus/`, `coralnpu/`, and `peripherals/` packages from test specs alone
- **4 cocotb TileLink tests**: TLUL2Axi protocol translation verified through Verilator simulation
- **0 E2E tests**: The top-level SoC didn't boot—AXI bus fabric worked but the scalar pipeline couldn't execute RISC-V instructions

The failure mode is consistent: **individual modules pass unit tests but system integration fails**. The CPU accepted memory transactions but never halted after firmware execution, hitting "timeout waiting for awready" on every E2E simulation.

3 Generalization: The harbor-task-gen Skill

To scale beyond Coral NPU, we built a reusable Claude Code skill (`harbor-task-gen`) that converts arbitrary RTL repositories into Harbor benchmark tasks. The skill guides the agent through:

1. Repository analysis (build system, test infrastructure, dependency mapping)
2. Skeleton creation (strip implementation, keep tests, handle submodules)
3. Dockerfile generation (2-stage or 3-stage based on cache persistence analysis)
4. Anti-cheat hardening (category-based artifact scrubbing)
5. Task configuration (proportional scoring, verifier, run infrastructure)
6. Sanity checking (unsolved≈0, solved=1.0)

The skill was iteratively refined through 4 rounds of evaluation against increasingly diverse repos, fixing 5 failure modes discovered during testing:

Bug	Root Cause
Dockerfile parse errors	Inline heredocs/Python in `RUN` commands (Docker uses `/bin/sh`)
pip install failures	Debian bookworm requires `--break-system-packages`
Missing submodules	Shallow clone doesn't init submodules; `warm_src` incomplete
Tool build failures	Building Verilator/Spike from source in Docker is fragile
Output token limit	Claude Code's 32K default too low for large RTL files

Table 2: Failure modes discovered during dynamic test runs and addressed in skill iterations.

4 Multi-Repo Benchmark Results

We applied the skill to 7 industry repositories spanning 5 build systems:

Task	Repo	Build	Tests	Docker	Sonnet Result
ibex-e2e	lowRISC Ibex	FuseSoC	4	OK	**1.000** (7.5 h)
ibex-full	lowRISC Ibex	FuseSoC	5	OK	0.000* (13.5 h)
secworks-aes-full	secworks/aes	FuseSoC	65	OK	0.000† (30 m)
veer-el2-block	VeeR EL2	Make	∼22	OK	0.000 (52 m)
coralnpu-full	Coral NPU	Bazel	305	OK	0.177 (10 h)
coralnpu-e2e	Coral NPU	Bazel	189	OK	0.098 (10 h)
veer-el2-full	VeeR EL2	Make	∼47	FAIL	Verilator version
caliptra-full	Caliptra	Make	53	FAIL	Commercial VIP
cva6-smoke	CVA6	Make+Bender	10	FAIL	Source build

Table 3: Benchmark results across repos. *Same RTL as ibex-e2e but infinite timer interrupt loop on 5th test (manually stopped). †Hit 32K output token limit.

4.1 The Ibex Result

The most significant finding: **Sonnet scored 1.000 on ibex-e2e**, writing a complete Ibex RISC-V core implementation (30 SystemVerilog files) from scratch in 7.5 hours. All 4 firmware simulation tests passed—`hello_test`, `dit_test`, `dummy_instr_test`, and `pmp_smoke_test`—meaning the agent produced a working RV32IMC pipeline with correct instruction fetch, decode, execute, load/store, CSR handling, and PMP.

The ibex-full task (5 tests, adding `tb_cs_registers`) used the *same RTL* but the agent's timer/CSR implementation had a bug: `timecmp` never advanced past `mtime`, creating an infinite IRQ→MRET cycle that produced 100 GB+ of debug trace logs over 8 hours before we manually stopped it.

5 Key Findings

1. **Cache exploitation is a real threat.** Without hardening, agents achieve inflated scores by reading cached build artifacts. The 0.958→0.098 drop on the same task demonstrates that most of the unhardened score was artifact copying, not genuine implementation.

2. **Models can write working RTL from test specs.** Sonnet's 1.000 on Ibex (30 files, working RISC-V core) and 25/25 Chisel unit tests on Coral NPU show that current models can implement individual modules correctly when given clear test specifications.

3. **System integration is the hard problem.** The consistent pattern across Coral NPU runs: unit tests pass, bus fabric works, but the CPU doesn't boot end-to-end. The gap between module-level correctness and system-level functionality is where agents currently fail.

4. **Build system diversity matters.** Of 12 task variants across 7 repos, 4 had Docker build failures due to tool version mismatches, commercial VIP dependencies, or fragile source builds. Real-world repos are messier than curated benchmarks.

5. **Agent behavioral issues affect scoring.** Agents exit voluntarily before using their full time budget (coralnpu finished in 5.7 h of a 24 h timeout), enable verbose debug tracing that generates unbounded log output, and hit output token limits on large file writes.

6 Infrastructure Developed

harbor-task-gen skill Reusable Claude Code skill for converting RTL repos into Harbor tasks. Tested on 7 repos, 5 build systems, 4 iterations of bugfixes. Handles Bazel, FuseSoC, Make, Bender, and filelist-driven flows.

Anti-cheat framework Three-category artifact classification (generated source, compiled objects, logs) applicable to any build system.

Auth infrastructure Credentials file mount with OAuth refresh token support for 24 h+ agent runs.

Monitoring pipeline Automated 20-minute trace monitoring with cheating detection, automatic commit-on-completion, and task queue pipelining.

12 benchmark tasks Across 7 repos: coralnpu-full/e2e, ibex-full/e2e, secworks-aes-full, veer-el2-full/block, cva6-full/smoke, caliptra-full/rtl-fw, pulp-common-cells-full.

7 Next Steps

1. **Fix remaining Docker builds.** Three repos have broken tasks:
   - CVA6: Need pre-built Verilator 5.x tarball instead of building from source
   - VeeR EL2 full: Investigate Verilator version requirements for nox block tests
   - Caliptra: Determine if Verilator flow can work without the commercial AXI VIP, or exclude those tests

2. **Scale to more repos.** The skill is ready for the remaining repos from our target list: OpenTitan (Bazel/DVSim), NVDLA (custom Make), OpenPiton (custom flow), CV32E40P (CORE-V-VERIF), PULP AXI (Bender).

3. **Multi-model comparison.** Run all working tasks with Opus, Sonnet, and Haiku to establish a model scaling curve on RTL benchmarks. Haiku scored 0.000 across all tasks; Sonnet scored 1.000 on Ibex — the gap suggests a sharp capability threshold.

4. **Deeper analysis of the Ibex result.** The 1.000 score deserves investigation: what did the agent's RTL look like? How close is it to the reference implementation? Did it use the same microarchitecture or invent a different one?

5. **Cross-pollination with Spec2GDSBench.** The finding that test depth determines difficulty (from Spec2GDSBench) aligns with our observation that unit tests are easy but E2E tests are hard. Investigate whether adding more fine-grained tests to RTL-Universe tasks would improve agent performance by providing better incremental feedback.

6. **Improve agent efficiency.** Current agents waste time on verbose debug tracing (100 GB logs), exit early without using their time budget, and hit output token limits. Investigate prompt engineering or Harbor adapter modifications to address these behavioral issues.

8 Repository Structure

```
rtl-universe/
 minrepro_task/           # 12 Harbor benchmark tasks
    coralnpu-full/      # Coral NPU, 305 tests (Bazel/Chisel)
    ibex-full/          # Ibex RISC-V, 5 tests (FuseSoC)
    secworks-aes-full/  # AES crypto, 65 tests (FuseSoC/Icarus)
    veer-el2-full/      # VeeR EL2, ~47 tests (Make/Verilator)
    ...                 # + 8 more variants
    tools/              # Sync scripts
 repos/                  # Cloned source repos (7 repos)
 jobs/                   # All Harbor run results with traces
 skills/
    harbor-task-gen/    # Reusable task generation skill
 BUGFIXES.md             # Issue tracker from dynamic runs
 reports/                # This report
```